

Yaseen Mohamed

✉ y24moham@uwaterloo.ca | ☎ +1 (647) 833-3952 | 🌐 github.com/y24moham | 📺 Yaseen Mohamed | 🌐 YaseenMohamed.netlify.app

Education

University of Waterloo | BAsC, Mechatronics Engineering **Average: 88.6%** Expected 2028
Internship Availability: Jan–Aug 2027 | 4 or 8 months
• **Relevant Coursework:** Automatic Control Systems, Microprocessor Systems & Interfacing, Real-Time Systems, Actuators & Power Electronics, Systems Modeling, Sensors & Instrumentation, Algorithms & Data Structures

Skills

- **Languages:** C, C++, Python, SQL, MATLAB, Bash
- **Embedded/Platforms:** STM32, ARM Cortex-M, RTOS, DMA, ADC, timers/interrupts, Arduino, Raspberry Pi
- **Interfaces/Debug:** CAN/CAN FD, Automotive Ethernet, I2C, SPI, UART; JTAG/SWD, ST-Link, OpenOCD, GDB
- **Software/Tools:** Linux, Git, BigQuery, Jira, Azure DevOps, Robot Framework, pytest, Pandas, NumPy
- **Hardware/CAD:** PCB design (Altium), soldering, oscilloscope, DMM, SolidWorks, Inventor, AutoCAD, FDM 3D printing

Experience

System Integration Engineer May – Aug 2026
Ford Motor Company CAN/CAN FD · Automotive Ethernet · C++ · Python · BigQuery/SQL
• Tested **OTA software-update scenarios** on a cloud-connected vehicle integration bench, configuring ECUs, flashing software versions, and analyzing **CAN/Ethernet logs** to reproduce field failures.
• Traced **gateway ECU C++ code and system logs** to identify OTA failure paths, then created investigation guides that cut recurring analysis from **1–2 hours to 5–15 minutes**.
• Investigated OTA issues across **1M–80M rows of connected-vehicle data** and built 6 Python tools to automate log analysis, reducing hands-on investigation time by **up to 90%**.
• Supported development and review of **software requirements for connected-vehicle features**, defining expected system behavior and edge cases to support implementation and validation.
• Optimized **BigQuery/SQL queries and workflows** for recurring vehicle-data investigations, reducing query runtime by **93%** and data-retrieval time from **2 hours to 10–15 minutes**.

Embedded Software Engineering Intern Sep – Dec 2025
Aversan Inc. STM32H7 · Embedded C · JTAG/OpenOCD · UART/I2C · Robot Framework
• Debugged and extended **STM32H7 embedded C firmware** for a safety-critical aerospace system under **DO-178 processes**, validating changes through bench testing.
• Built **5 Python/UART tools** to automate firmware debugging and verification, uncovering dozens of firmware/test defects and reducing test time by **70%**.
• Built a **Linux-based Python/OpenOCD JTAG boundary-scan tool** to detect soldering, short-circuit, and connectivity faults before firmware bring-up.
• Implemented **STM32H7 peripheral test firmware** using bit-banged I2C, DMA, timers, and interrupt-driven UART, and automated ATE verification with Robot Framework, reducing component test time by **90%**.

Engineering Research Assistant Jan – Apr 2025
University of Waterloo School of Optometry Arduino/C++ · Raspberry Pi/Python · PCB design · SolidWorks/3D printing
• Built a **48-channel pressure-based footstep-detection system** using Arduino, a custom PCB, and Raspberry Pi, and developed a decay-compensated C++/Python acquisition pipeline that reduced sampling interval from **3 s to 3 ms**.
• Developed a **Python preprocessing/visualization GUI** with a sensor-grouping algorithm and designed SolidWorks/FDM equipment upgrades that reduced setup time by **80%**.

Projects

STM32 2-Axis Motion Controller | [GitHub](#) · [Demos](#) | C, STM32, SPI, ADC, UART, Python Jan – Apr 2026
• Built an **STM32-based dual-axis motion-control system** for independent stepper-motor control, using SPI motor drivers, ADC speed inputs, and keyboard commands from a Python GUI over UART.
• Implemented **interrupt-driven limit-switch protection** with wiring-fault detection and direction-aware motion blocking, and resolved an **ADC/EXTI race condition** using persistent axis states.

Quadcopter Prototype | [GitHub](#) · [Demo](#) | C++, Arduino, IMU, Timers, PID Control Jun – Aug 2020
• Prototyped a **real-time PID attitude-control system** in C++, using IMU gyroscope feedback and custom PCB hardware to implement and tune closed-loop stabilization on a test rig.